Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 01	
Title of LAB Practical: Basic Python Programs.			
DOP: 01/10/2026		**DOS:** 05/10/2026	
CO Mapped: CO1	**PO Mapped:** PO3, PO4, PSO1, PSO2	**Faculty Signature:**	**Marks:**

Name: Vikas Kushwaha **Roll No: 29/B**

Practical 1

Aim: Basic Python Programs.

Theory:

The programs covered follow the foundational workflow of Python programming: receiving data, applying mathematical or logical transformations, and displaying the results.

1. Input and Output Mechanisms

input() function: Reads data from the keyboard. The value returned by input() is always stored as a string (str).

```
name = input("Enter your name: ")
```

Explicit type casting: Converts input strings into suitable numerical data types.

```
age = int(input("Enter your age: "))
price = float(input("Enter the price: "))

int() converts a value into an integer.
float() converts a value into a decimal number.
```

print() function: Displays text, variables, and calculated results.

```
print("Total =", total)
```

Formatted strings: F-strings allow variables and expressions to be inserted directly into a string.

```
print(f"The sum of the numbers is {total}")
```

Expressions can also be included:

```
print(f"Area = {0.5 * base * height}")
```

2. Variables and Assignment Operations

Dynamic typing: Python variables do not require explicit type declarations. Their types are determined automatically at runtime.

```
number = 10        # Integer
amount = 25.75     # Float
message = "Hello"  # String
```

The same variable can later refer to a value of another type:

```
value = 10
value = "Python"
```

Assignment operator: The = operator assigns a value to a variable.

```
total = a + b
```

Temporary storage: Two values can be swapped using an intermediate variable.

```
temp = a
a = b
b = temp
```

Here, temp temporarily stores the original value of a.

Tuple unpacking: Python also supports simultaneous assignment.

```
a, b = b, a
```

The values on the right-hand side are evaluated first and then assigned to the variables on the left-hand side. This allows two values to be swapped without using an additional variable.

3. Arithmetic and Formula Implementation

Python provides standard arithmetic operators for performing calculations.

```
Operator   Operation       Example
+          Addition        a + b
-          Subtraction     a - b
*          Multiplication  a * b
/          Division        a / b
**         Exponentiation  a ** 2
```

Addition of three numbers:

```
total = a + b + c
```

Area of a triangle:

Area = ½ × base × height

In Python:

```
area = 0.5 * base * height
```

4. Conditional Statements

Conditional statements allow a program to make decisions based on whether a condition is true or false.

```
if a == 0:
    print("This is not a quadratic equation.")
else:
    print("This is a quadratic equation.")
```

Common comparison operators include:

```
==     equal to
!=     not equal to
>      greater than
<      less than
>=     greater than or equal to
<=     less than or equal to
```

Python uses indentation to define the body of an if or else block.

5. Low-Level and Binary Computations

Bitwise Manipulation

Bitwise operators work directly on the binary representation of integers.

The main bitwise operators are:

```
Operator   Name  Description
&          AND   Sets a bit to 1 if both corresponding bits are 1
`          `     OR
^          XOR   Sets a bit to 1 if the corresponding bits are
different
~          NOT   Inverts the bits
<<         Left shift      Moves bits to the left
>>         Right shift     Moves bits to the right
```

For example:

```
a = 5
b = 3

print(a & b)
print(a | b)
print(a ^ b)
print(~a)
print(a << 1)
print(a >> 1)
```

Integers are represented internally using binary notation. Negative integers are generally represented using two's complement notation.
Base Conversion

Binary to decimal: The int() function can convert a string from a specified base into an integer.

```
decimal_number = int("1010", 2)
```

Here, the argument '2' specifies base 2.

Decimal to binary: The bin() function converts an integer into a binary string.

```
binary_number = bin(10)
```

The result is:

```
0b1010
```

The prefix 0b identifies the value as binary. It can be removed using string slicing:

```
binary_number = bin(10)[2:]
```

6. Standard Library Integration

Python includes a standard library containing pre-built modules for common programming tasks. Modules can be imported using the import statement.
cmath Module

The cmath module performs mathematical operations involving complex numbers.

```
import cmath
```

It is useful when solving quadratic equations because the discriminant may be negative.

```
root = cmath.sqrt(discriminant)
```

Unlike ordinary square-root functions, cmath.sqrt() can calculate the square root of a negative number and return a complex result.
random Module

The random module generates pseudo-random values.

```
import random
```

The function:

```
random.randint(0, 100)
```

generates a random integer between 0 and 100, including both endpoints.

The values are called pseudo-random because they are generated by a deterministic algorithm, although they appear random for ordinary programming purposes.
calendar Module

The calendar module is used to display calendar information.

```
import calendar
```

The function:

```
calendar.month(2019, 1)
```

displays the calendar for January 2019. The first argument represents the year, while the second argument represents the month number.

7. Strings and Character Data

A string is a sequence of characters enclosed in single or double quotation marks.

```
binary_number = "1010"
```

Although it contains digits, "1010" is a string because it is enclosed in quotation marks.

String slicing can extract part of a string:

```
value = "0b1010"
binary_digits = value[2:]
```

The expression value[2:] extracts the string from index 2 to the end, producing:
text

```
1010
```

8. Comments and Program Readability

Comments provide explanations within a program. They begin with the # symbol and are ignored by Python during execution.

```
# Calculate the area of the triangle
area = 0.5 * base * height
```

Comments improve the readability and maintainability of a program.

9. General Program Structure

Most of the programs use the following input-process-output model:

Input: Accept values from the user.
Processing: Perform calculations, comparisons, or conversions.

Output: Display the final result.

Example:

```
a = float(input("Enter the first number: "))
b = float(input("Enter the second number: "))

result = a + b

print(f"The sum is {result}")
```

This structure forms the basis of many beginner-level Python programs.

Programs:

1. Add Three Numbers

CODE

```
# Add Three Numbers
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

total = a + b + c

print("Sum =", total)
```

OUTPUT:

```
Enter first number: 12
Enter second number: 24
Enter third number: 36
Sum = 72.0
```

2. To Swap two No using third variable and without using third variable.

CODE

```
# Swap Two Numbers
# Using a third variable

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

temp = a
a = b
b = temp

print("After swapping:")
print("a =", a)
print("b =", b)

# Without using a third variable

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

a, b = b, a

print("After swapping:")
print("a =", a)
print("b =", b)
```

OUTPUT:

```
Enter first number: 25
Enter second number: 45
After swapping:
a = 45
b = 25
```

```
Enter first number: 12
Enter second number: 22
After swapping:
a = 22
b = 12
```

3. Calculate area of triangle

CODE

```
# Calculate the Area of a Triangle

base = float(input("Enter the base of the triangle: "))
height = float(input("Enter the height of the triangle: "))

area = 0.5 * base * height

print("Area of triangle =", area)
```

OUTPUT:

```
Enter the base of the triangle: 10
Enter the height of the triangle: 20
Area of triangle = 100.0
```

4. To Solve Quadratic equation

CODE

```
# Solve a Quadratic Equation

# For an equation of the form:

# ax2+bx+c=0
# ax2+bx+c=0

import cmath

a = float(input("Enter coefficient a: "))
b = float(input("Enter coefficient b: "))
c = float(input("Enter coefficient c: "))

if a == 0:
```

CODE

```
    print("This is not a quadratic equation.")
else:
    discriminant = b**2 - 4*a*c

    root1 = (-b + cmath.sqrt(discriminant)) / (2*a)
    root2 = (-b - cmath.sqrt(discriminant)) / (2*a)

    print("Root 1 =", root1)
    print("Root 2 =", root2)
```

OUTPUT:

```
Enter coefficient a: 1
Enter coefficient b: 2
Enter coefficient c: 4
Root 1 = (-1+1.7320508075688772j)
Root 2 = (-1-1.7320508075688772j)
```

5. To use Bitwise operators

CODE

```
# Use Bitwise Operators

a = int(input("Enter first integer: "))
b = int(input("Enter second integer: "))

print("a & b =", a & b)      # Bitwise AND
print("a | b =", a | b)      # Bitwise OR
print("a ^ b =", a ^ b)      # Bitwise XOR
print("~a =", ~a)            # Bitwise NOT
print("a << 1 =", a << 1)    # Left shift
print("a >> 1 =", a >> 1)    # Right shift
```

OUTPUT:

```
Enter first integer: 10
Enter second integer: 20
a & b = 0
a | b = 30
a ^ b = 30
~a = -11
a << 1 = 20
a >> 1 = 5
```

6. To compute compound interest given all the required values.

CODE

```
# Calculate Compound Interest

# Formula:

# A=P(1+r100n)nt
# A=P(1+100nr)nt

# CI=A-P
# CI=A-P

principal = float(input("Enter principal amount: "))
rate = float(input("Enter annual interest rate (%): "))
time = float(input("Enter time in years: "))
frequency = int(input("Enter number of compoundings per year: "))

amount = principal * (1 + rate / (100 * frequency)) ** (frequency *
time)
compound_interest = amount - principal

print("Final amount =", amount)
print("Compound interest =", compound_interest)
```

OUTPUT:

```
Enter principal amount: 10000
Enter annual interest rate (%): 10
Enter time in years: 2
Enter number of compoundings per year: 1
Final amount = 12100.000000000002
Compound interest = 2100.000000000002
```

7. To generate a random number between 0 and 100

CODE

```
# Generate a Random Number Between 0 and 100

import random

number = random.randint(0, 100)

print("Random number =", number)
```

OUTPUT:

```
Random number = 68
```

8. To display calendar for the January 2019

CODE

```
# Display the Calendar for January 2019

import calendar

print(calendar.month(2019, 1))
```

OUTPUT:

```
    January 2019
Mo Tu We Th Fr Sa Su
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31
```

9. To add two binary numbers

CODE

```
# Add Two Binary Numbers

binary1 = input("Enter the first binary number: ")
binary2 = input("Enter the second binary number: ")

number1 = int(binary1, 2)
number2 = int(binary2, 2)

sum_binary = number1 + number2

print("Sum in binary =", bin(sum_binary)[2:])
```

OUTPUT:

```
Enter the first binary number: 101
Enter the second binary number: 110
Sum in binary = 1011
```

Conclusion:

The programs presented demonstrate several fundamental features of Python programming, including input and output operations, variables, dynamic typing, type casting, arithmetic calculations, conditional statements, bitwise operations, binary conversion, and the use of standard library modules. They also show how mathematical formulas can be translated into Python expressions to solve practical problems such as calculating the area of a triangle, finding compound interest, and solving quadratic equations.

These programs follow the basic input-process-output model: data is accepted from the user, processed using mathematical or logical operations, and then displayed as output. Features such as tuple unpacking, formatted strings, module integration, and string manipulation make Python concise and readable. Understanding these concepts provides a strong foundation for developing more advanced Python programs involving functions, loops, data structures, file handling, and object-oriented programming.